University Records System — Team Roadmap

A shared, phase-by-phase plan for the whole team: **Backend** (FastAPI), **Web** (React), and **Mobile** (Kotlin). Everyone is new to their stack — the goal is to move fast but stay in sync, meeting up after each phase to integrate real screens with the real API.

---

1. Team structure

Team	Stack	Owns
Backend	FastAPI + PostgreSQL (Docker)	API, database, business rules, auth
Web	React	Browser UI
Mobile	Kotlin (Android)	Android app UI, biometric auth

Golden rule for staying in sync: the backend's auto-generated docs at `http://127.0.0.1:8000/docs` are the *contract*. Whenever backend adds or changes an endpoint, that page updates automatically — frontend teams should treat it as the source of truth for what fields exist, what's required, and what a response looks like. This means frontend can start against **dummy/mock data shaped exactly like that contract**, then swap in the real `fetch` /Retrofit call later with minimal changes.

---

2. The domain: what we're building

A small university records system. Four entities:

Entity	Fields (starting point — refine as we go)
Staff	id, first_name, last_name, email, phone_number, gender (enum) — the person capturing records, and later, who logs in
Student	**reg_number (primary key)**, first_name, last_name, email, phone_number, gender (enum), faculty_id, program_id, intake_year — created **only** through the multipart wizard, never a flat form

Entity	Fields (starting point — refine as we go)
Faculty	id, code (e.g. `SCI`), name (e.g. "Faculty of Science")
Program	id, code (e.g. `BSCS`), name, faculty_id (belongs to a Faculty)
Mark	id, student_reg_number (FK → Student.reg_number), course_code, course_name, score, semester (enum) — entered independently, not part of registration

> **reg_number as primary key — a design trade-off worth understanding:** most systems use an auto-increment `id` as the primary key and keep "natural" identifiers like a registration number as a separate unique field, because natural keys occasionally need to change (typos, policy changes) and changing a primary key that other tables reference (like `Mark.student_reg_number`) is awkward. We're deliberately using `reg_number` as the real primary key anyway here, because (a) it's genuinely immutable once assigned in a real university, and (b) it doubles as the student's login identifier in Phase 9 — a nice real-world payoff for the simpler design. Captured as a required field on the Bio Data form (Phase 1) and enforced unique by the database itself (duplicate → `409 Conflict`).

A student's *full* record is captured in **two** phases via the multipart wizard:

- **Phase 1 — Bio data**: first_name, last_name, email, phone_number, gender
- **Phase 2 — Uni info**: faculty (selector) → program (selector, filtered by chosen faculty), intake year

Marks are **not** part of registration — they're entered later, on their own independent screen (Phase 7 below), because in a real university marks get entered repeatedly (every semester) long after a student is registered once.

> **Note:** the Staff quick-registration form (Phase 2) and the Student's Phase 1 (Bio data) use the exact same shape of fields (name/email/phone/ gender) — but they're two different entities. You'll literally build this shape of form twice: once flat for Staff, once as step 1 of the Student wizard. That repetition is what makes the wizard's extra plumbing (state across steps, "Next" navigation) obvious by contrast.

---

3. Phase-by-phase plan

Each phase lists Backend / Web tasks side by side, plus the **meeting point** — the moment both sides plug into each other for real.

Phase 0 — Environment setup (all teams, in parallel)

Backend	Web
Already done — see SETUP_GUIDE.md (FastAPI + Docker + Postgres)	Install Node.js LTS, scaffold a React app (see §6 below)

Meeting point: none yet — pure setup.

Phase 1 — Modular project structure

Backend	Web
Split `main.py` into modules: `routers/`, `schemas/` (Pydantic), `models/` (SQLAlchemy), `services/` (business logic), `database.py`. One router per entity (`students.py`, `faculties.py`, `programs.py`)	Set up folder structure: `components/`, `pages/` (or `routes/`), `api/` (a thin client wrapper), `types/`

Meeting point: none — internal scaffolding on both sides.

Phase 2 — Simple POST: Staff registration

Backend	Web
`POST /staff` — Pydantic schema validates first_name, last_name, email, phone_number, gender (a `Gender` enum — see §4). Returns `201 Created` with the new staff record, or `422` on bad input	**First, build it dummy:** a single form screen with client-side validation (required fields, email format, phone format, gender as a dropdown) that just logs the payload to console
Add `GET /staff/{id}` to fetch one back	Once backend's `/staff` is live, swap the dummy submit for a real `fetch`/axios POST call. Show success/error based on the real response

Meeting point: ✅ Web's staff registration form POSTs to the real backend and shows the created staff record back on screen.

(Mobile does the same thing in Kotlin — see §5.)

> Staff are the ones who log in and capture student data — this entity is what Phase 9 (auth) builds directly on top of.

Phase 3 — Faculty & Program (introduces the selector pattern)

Backend	Web
`POST /faculties`, `GET /faculties` (returns a list — this is what feeds selectors). `POST /programs` (body includes `faculty_id`), `GET /programs?faculty_id=` (filtered list)	Dummy Faculty form (code, name) → wire to `POST /faculties`. Dummy Program form with a `<select>` populated by `GET /faculties`, then submits to `POST /programs`

Meeting point: ✅ Web's Program form loads real faculties into a dropdown, and creates a program tied to the selected faculty.

Phase 4 — GET with pagination & query parameters

Backend	Web
`GET /students?page=1&page_size=20` — return `{items, total, page, page_size}`. Also support filtering, e.g. `GET /students?gender=female`	Build a paged list/table screen: shows current page of students, Next/Previous buttons, a filter input tied to a query param

Meeting point: ✅ Web's list screen renders real, paged data and filters update the query correctly.

This is also where you deliberately test **query parameter validation** — try `page=-1`, `page_size=0`, `page_size=9999` and see what your backend should do about it (reject with `422`, or clamp?).

Phase 5 — PUT, PATCH, DELETE

Backend	Web
`PUT /students/{reg_number}` (full replace — all fields required), `PATCH /students/{reg_number}` (partial update — only sent fields change), `DELETE /students/{reg_number}` (`204 No Content` on success, `404` if not found)	Edit screen (pre-filled form → `PUT` or `PATCH`, discuss which one and why), Delete button with a confirmation dialog

Meeting point: ✅ Web can edit and delete a real student record.

This phase is a great place to deliberately study status codes: what happens (and what *should* happen) when you `DELETE` something twice, or `PATCH` an id that doesn't exist?

Phase 6 — Multipart wizard: student registration (2 steps)

Backend	Web
`POST /students` (bio data, including `reg_number` — same shape as Phase 2's Staff form plus the reg number), `PATCH /students/{reg_number}/university-info` (sets faculty_id/program_id/intake_year)	Two-step form component: step state persists across steps, "Back"/"Next" navigation, step 2 uses the faculty→program selector from Phase 3

Meeting point: ✅ A staff user can fully register a student across both screens, ending up as one complete record in the database.

No transaction needed yet — two separate requests, two separate commits, and that's fine, because a student existing with incomplete uni-info isn't a broken state (you can always come back and finish it later). Transactions become necessary in the *next* phase.

Phase 7 — Marks entry (independent, selector-driven)

Backend	Web
Reuse the paged/search list from Phase 4 (`GET /students?search=`) to power a student picker. `POST /students/{reg_number}/marks` — accepts a **list** of mark rows (course_code, course_name, score, semester enum)	A dedicated "Enter Marks" screen: search/select an already-registered student (typeahead or paged selector), then a small

Backend	Web
in one request, wrapped in **one database transaction**: all rows save, or none do	repeatable row form ("add another mark") before one final submit

Meeting point: ✅ A staff user can pick any already-registered student and submit a batch of marks for them, independent of when that student was registered.

This is where **transactions** become concrete: if row 2 of the batch has a bad score, the whole batch should roll back rather than leaving row 1 saved and row 2 missing. Ask "what should the user see if this happens?" — that's a real error-handling design question.

Phase 8 — Images & logos

Backend	Web
A file upload endpoint (e.g. `POST /students/{reg_number}/photo`, multipart/form-data), store the file (start with local disk under a `static/` folder, serve via `StaticFiles`), save just the path/URL in the DB	An image upload input with a live preview before submit, and display uploaded photos/logos on the list & detail screens

Meeting point: ✅ A photo uploaded from the browser is stored by the backend and displayed back correctly.

Phase 9 — Authentication (Staff and Student, two roles)

There are now two kinds of logins, sharing one login endpoint:

- **Staff** log in with **email + password**
- **Student** log in with **reg_number + password** (this is the payoff of making `reg_number` the primary key — it's already the natural, memorable login identifier). Add a `password_hash` field to Student for this; simplest approach is staff sets an initial password for the student right on the Bio Data form (Phase 1), same as any other field.

Backend	Web
Hashed passwords (`passlib`) on both `Staff` and `Student` . `POST /auth/login` accepts an `identifier` (email or reg_number) + password, checks both tables, and returns a JWT containing `sub` (the user's id/reg_number) and a `role` claim (`"staff"` or `"student"`)	Login screen, store the token (discuss: localStorage vs. httpOnly cookie, and why), attach it to every API call, redirect on `401`

Meeting point: ✅ Both a staff member and a student can log in through the same screen and each get a token carrying their role.

(Mobile adds biometric login on top of this same JWT flow — see §5.)

Phase 10 — RBAC: role- and ownership-based permissions

Now that both roles can log in, lock down what each is actually allowed to do — and add the student's own "My Marks" screen.

Backend	Web
A `require_role(*roles)` FastAPI dependency that reads the `role` claim from the JWT and raises `403` if it doesn't match. Staff-only routes (create/edit/delete anything) use `require_role("staff")` . The new `GET /me/marks` route uses `require_role("student")` **and** filters `WHERE student_reg_number = current_user.reg_number` — a student's token can only ever return *their own* marks, never anyone else's, no matter what they pass in a URL	A dedicated read-only **"My Marks"** screen for logged-in students — no selector needed, it just calls `GET /me/marks` using their own token. Also: hide/disable staff-only nav items and buttons when the logged-in user's role is `"student"`

Meeting point: ✅ A student logs in and sees only their own marks; a staff member logs in and sees the full staff toolset. Neither can reach the other's screens.

The one rule that matters most in RBAC: *hiding a button on the frontend is a UX nicety, not security.* The backend must independently enforce every permission check — assume any request could come from someone who bypassed your UI entirely (Postman, curl, a modified mobile APK). Two distinct checks, both required:

- **Role check** — is this user's role even allowed to call this endpoint at all? (`require_role("staff")`)

- **Ownership check** — for data tied to a specific person (marks, profile), does the logged-in user's own id match the record being requested, or are they staff (who can see anyone's)? A role check alone isn't enough here — without the ownership filter, any authenticated student could read any *other* student's marks just by knowing their reg_number.

---

4. Backend good practices (apply from Phase 1 onward)

- **Separate layers**: `router` (HTTP concerns only) → `service` (business logic) → `model` (DB shape). Don't put SQL queries directly in route functions once the app grows past a couple of endpoints.
- **Pydantic schemas ≠ SQLAlchemy models.** Have an input schema (`StudentCreate`), an output schema (`StudentOut`), and the DB model — keeps what clients can send separate from what's stored.
- **One router per entity**, mounted with a prefix: `app.include_router(students.router, prefix="/students")` . This is literally what "splitting into modules so one failing part doesn't break another" means in FastAPI — a bug in the marks module shouldn't be able to crash the faculties module, because they're separate files with separate routers.
- **Use response_model** on every route (`@app.get("/students", response_model=list[StudentOut])`) — FastAPI will strip out any field you didn't mean to expose (e.g. a password hash).
- **Transactions**: SQLAlchemy sessions default to "commit only when you say so." For multi-step writes (Phase 6), do all the inserts, then one `db.commit()` at the end; if anything raises, the session should `db.rollback()` (a `try/except` around the whole operation, or FastAPI dependency-level session handling that rolls back on exception).
- **Validate at the boundary.** Pydantic already rejects malformed input before your function body even runs — don't re-validate the same thing deeper in the service layer.
- **Status codes are part of your API design, not an afterthought.** Decide up front: `201` for creation, `200` for reads/updates, `204` for delete, `404` for missing resources, `422` for validation errors, `409` for conflicts (e.g. duplicate email).
- **Use enums for every fixed-choice field**, not free-text strings — `gender` and `semester` are the first two, and later `staff_role` / `student_status` will be too. Define them once in Python:

```
from enum import Enum


class Gender(str, Enum):
    MALE = "male"
```

```
    FEMALE = "female"
    OTHER = "other"


class Semester(str, Enum):
    SEM_1 = "sem_1"
    SEM_2 = "sem_2"
```

Use the type directly in your Pydantic schema (`gender: Gender`) — FastAPI then rejects any other value with a `422` automatically, *and* renders the valid options as a dropdown in `/docs` . Use the same enum as the SQLAlchemy column type (`Column(SQLEnum(Gender))`) so Postgres enforces it too, not just the API layer. This is also exactly what makes frontend selectors trivial: expose the enum's values (e.g. via a small `GET /enums/gender` helper endpoint, or just hardcode the known set on the frontend since it rarely changes) and a `<select>` /dropdown falls right out of it — the same pattern as the faculty/program selector, just with a fixed list instead of one fetched from the database.

---

5. Frontend good practices

Web (React)

- **A thin API layer.** One `api/students.js` (or `.ts`) module wrapping `fetch` /axios calls — components never call `fetch` directly. Makes it trivial to swap dummy data for real calls later.
- **Form validation library** — `react-hook-form` + `zod` (or `yup`) is the standard combo. Define the validation schema once, reuse it for both the simple form and each step of the wizard.
- **Environment variables** for the API base URL (`VITE_API_URL` if using Vite), never hardcode `http://127.0.0.1:8000` in components.
- **Loading / error / empty states** for every screen that fetches data — a paged list screen has at minimum 4 states: loading, error, empty, populated. Design for all of them from day one.
- **RBAC on the frontend (Phase 10):** decode the JWT's `role` claim once at login and keep it in your auth context/state. Use it for two things only — (1) **route guards**: redirect away from staff-only pages if the logged-in role is `"student"` , and (2) **hiding UI**: don't render buttons/links to actions a role can't perform. Treat this as UX polish only — the backend's role and ownership checks are what actually keep data safe, not this.

Mobile (Kotlin) — brief pointers, since it mirrors web conceptually

- **Retrofit** for the API client, **Jetpack Compose** for screens (modern standard, easier to learn than the older XML/View system).
- **ViewModel + StateFlow** to hold screen state (loading/error/data) — the Compose equivalent of React's state + API layer pattern.
- **Biometric login**: use Android's `BiometricPrompt` API *after* a normal JWT login has happened once — biometrics typically unlock a securely-stored token (via `EncryptedSharedPreferences` or the Android Keystore), it doesn't replace the backend auth flow.
- **RBAC**: same principle as web — read the `role` claim to decide which screens/nav destinations are reachable, but never treat that as the actual security boundary.

---

6. Environment setup guides

Web team (React)

1. Install Node.js LTS (includes `npm`).
2. Scaffold with Vite (faster and simpler than Create React App):

```
npm create vite@latest university-web -- --template react
cd university-web
npm install
npm run dev
```

3. Install form + HTTP helpers:

```
npm install react-hook-form zod @hookform/resolvers axios
```

4. Confirm it's running at `http://localhost:5173`, then try calling the backend's `/health` endpoint from a component to prove cross-origin requests work (you'll need to add CORS middleware on the FastAPI side — ask backend to add `fastapi.middleware.cors.CORSMiddleware` allowing `http://localhost:5173`).

Mobile team (Kotlin)

1. Install **Android Studio** (includes the Kotlin toolchain and an emulator manager).

2. New Project → "Empty Activity" template with **Jetpack Compose** checked.
3. Add dependencies (in `build.gradle.kts`, app module): Retrofit, OkHttp logging interceptor, and (later) `androidx.biometric`.
4. Run the emulator, confirm you can hit `http://10.0.2.2:8000/health` (the emulator's special alias for your host machine's `localhost`) and see a response.

Backend team

Already fully set up — see SETUP_GUIDE.md. One addition needed for this phase: enable CORS so React (and later the emulator) can call the API from a different origin:

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173", "http://10.0.2.2:8000"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

---

7. HTTP status code cheat sheet (build this up as you hit real cases)

Code	Meaning	When you'll see it here
200	OK	Successful GET, PUT, PATCH
201	Created	Successful POST that creates a resource
204	No Content	Successful DELETE
400	Bad Request	Malformed request the server can't parse at all
401	Unauthorized	Missing/invalid auth token (Phase 9)
403	Forbidden	Valid token, but wrong role or not the record's owner (Phase 10 RBAC)

Code	Meaning	When you'll see it here
404	Not Found	GET/PATCH/DELETE on an id/reg_number that doesn't exist
409	Conflict	e.g. duplicate reg_number or email on registration
422	Unprocessable Entity	Pydantic validation failed (FastAPI's default for bad input shape)
500	Internal Server Error	An unhandled bug — treat every one you see as something to fix, not ignore

---

8. Suggested order to actually start

1. Backend: Phase 1 (modular restructure) — small, low-risk, sets up everything after it.
2. Backend + Web in parallel: Phase 2 (simple POST) — first real meeting point, builds confidence fast.
3. From there, follow the phases in order. Don't skip ahead to auth or multipart before PUT/PATCH/DELETE/pagination feel comfortable — those fundamentals get reused in every later phase.

Ready to start on Phase 1 (splitting `main.py` into modules) whenever you are — that's the natural next backend step from where we left off.